Junior Java Backend fejlesztő IV.

OOP II.

Botlik Béla

Öröklődés, polimorfizmus, absztrakt osztály, kivételkezelés

(A prezentációban felhasználásra kerültek diák a Ruander Oktatóközpont korábbi tananyagából. Szerzők: Béres Gábor, Czigléczky Gábor, Fehér Róbert, Kis Balázs)

Öröklődés I.

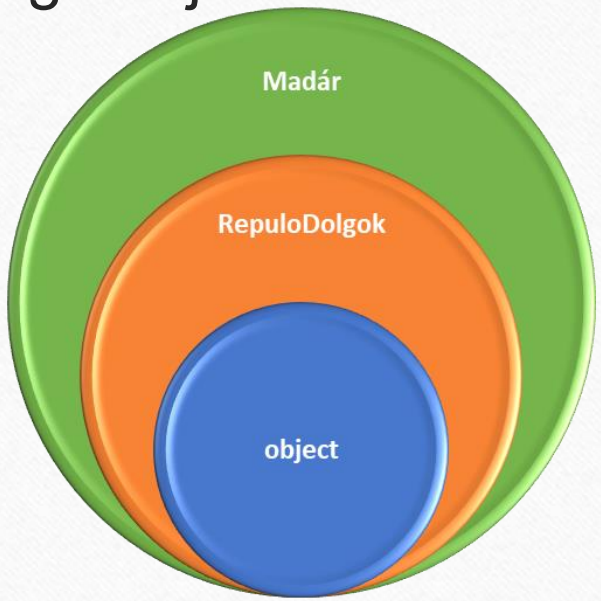
- Ahogy már az OO alapelveknél tanultuk, az öröklődés egy specializálódási folyamat, mely során egy osztályból származtatunk egy másik osztályt, mely valamilyen speciális feladatot tud ellátni, azonban témakörét tekintve az őséhez tartozik.
  - Repülő dolgok → Madár
  - Növények → Zöldségek → Paprika

Öröklődés II.

- Az öröklődés feladatai:
  - Ne egy osztály végezzen el mindent, ami egy fő témakörbe tartozik.
  - Leegyszerűsítse a program aktuális létrehozását, illetve későbbi bővítését az adott ősből specializált osztályok létrehozásával.
    - Minden egyes létrejött osztályból létrehozott objektum továbbra is őstípusként kezelhető marad (Polimorfizmus).
      - `RepuloDolgok obj = new Madar();`

Öröklődés III.

- Mint azt már tudjuk, a this segítségével elérjük az aktuális objektumot, a super segítségével pedig elérjük az ősosztályt.
- Örököltetés esetén a leszármazott osztályból létrejött objektum tartalmaz egy ősosztályból létrejött objektumot:



Öröklődés IV.

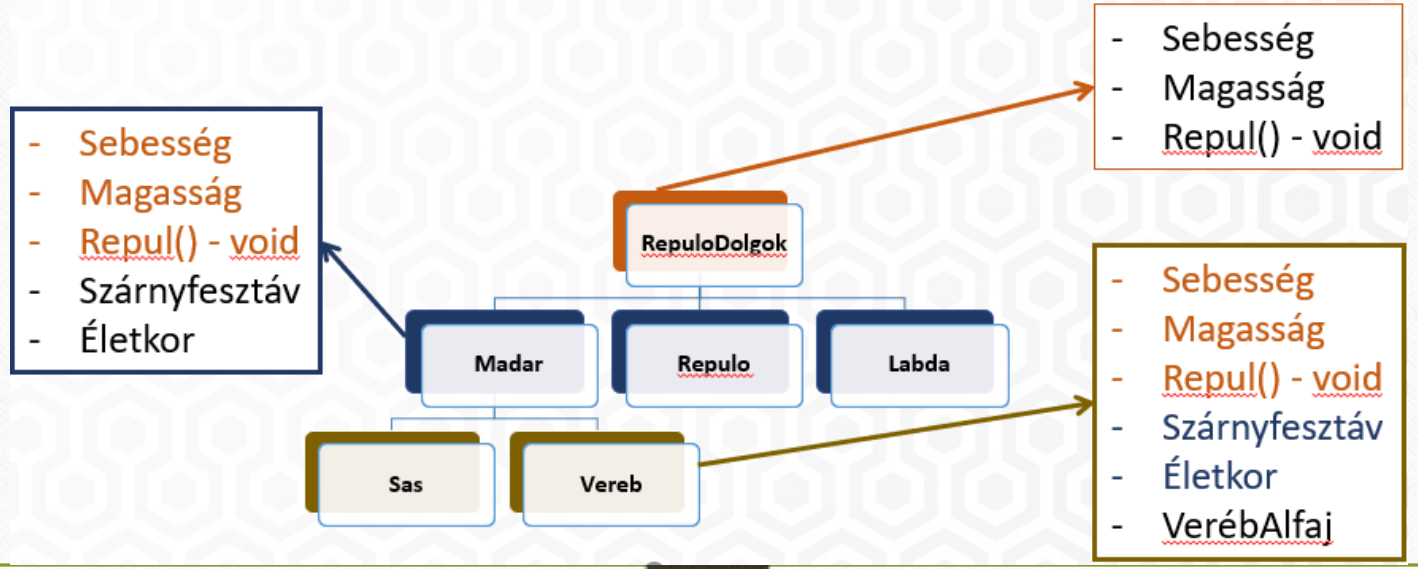
- Amire öröklődéskor oda kell figyelnünk:
  - Láthatósági szintek: már az ősosztály létrehozásakor gondoskodhatunk arról, hogy a leszármazott osztály elérje az ős osztályból az egyes részegységeket.
    - protected → a leszármazott másik csomagból (package) is eléri az ősének elemeit.
- Konstruktor felépítése: mivel egy specializált utód jön létre egy osztályból, annak implementálnia kell az ősét is, így annak konstruktorát is meg kell hívja.
  - Általános hiba, hogy miután a paraméteres konstruktor nem fut le paraméter nélkül, és így a Java hibát jelez, a program írója létrehoz egy paraméter nélküli konstruktort, hogy lefusson a program.

Öröklődés V.

- Megvalósítása:
  - `class Madar extends RepuloDolgok{...}`
  - Ha az ősnek nincs paraméter nélküli konstruktora, akkor a leszármazott osztály konstruktorából meg kell hívni:
    - public Madar(<paraméterek>)
      {
      super(<paraméterek>);
      //…
      }

Öröklődés VI.

- Az öröklődés alapvető feladata, hogy megkönnyítse a fejlesztés menetét oly módon, hogy a már megírt osztályokból származtatva azok specifikus változatát, megspórolhatjuk a külön fejlesztési időt:

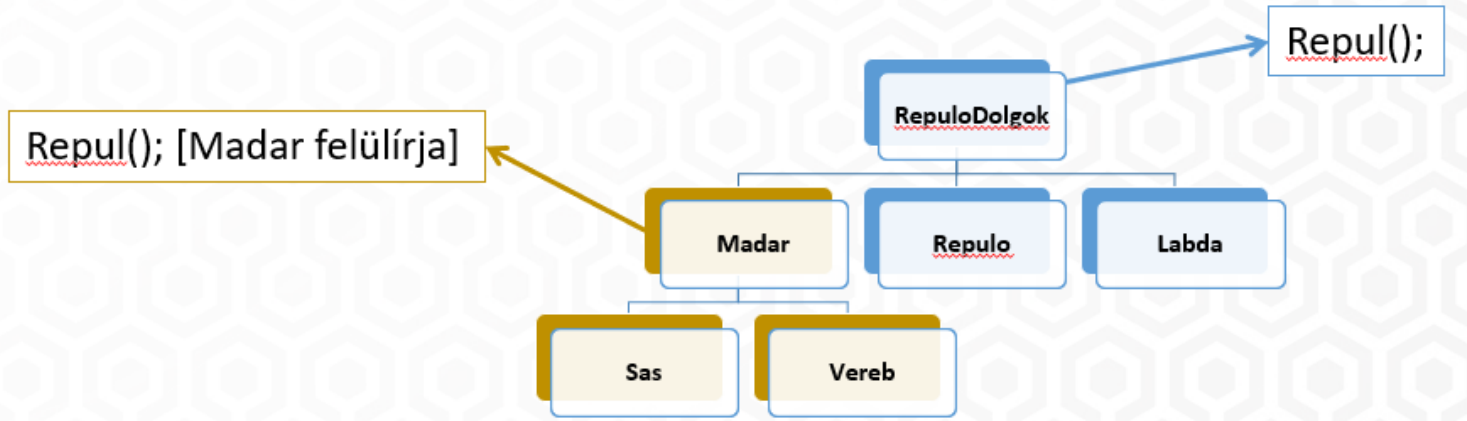


Alprogram felülbírálás I.

- Egyes esetekben előfordulhat, hogy a származtatáshoz használt ősosztály valamilyen alprogramot implementál magában, azonban a leszármazottak ugyanazon alprogramot másképpen kell végrehajtsák.
- Ilyen esetekben felül lehet írni azt az alprogramot, melyet az elődd megszabott.

Alprogram felülbírálás II.

- Az alprogramok felülbírálása továbbá hatással van a hierarchia további elemeire is:



  - Ha a fentebbi hierarchiában a RepuloDolgoknak van egy olyan alprogramja, melyet a Madar felül is ír, a Sas és a Vereb, már a Madar által felülírt alprogramot futtatja, amennyiben azok nem írják felül újra azt.

Feladat I.

- Készítsünk programot, mely egy vegyeskereskedés (ABC) alapszintű raktárkezelését látja el. Alapvetően kétféle termékkategóriát kell tároljunk:
  - Tejtermék
    - Zsírtartalom
  - Pékáru
    - Teljeskiőrlésű-e
  - Minden termékről tudjuk a következőket:
    - Megnevezés
    - Cikkszám
    - Ár
    - Darabszám
- Minden olyan termék, ami nem tartozik egyik kategóriába sem, általános termékként legyen tárolva. Hozzuk létre az adatszerkezetet és egy-egy objektumot minden típushoz.

Absztrakt osztály I.

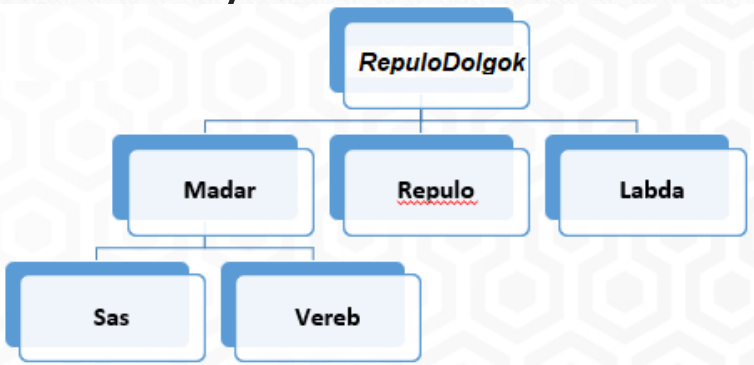
- Absztrakt osztály, egy olyan speciális osztály, mely kizárólagosan örököltetésre, illetve a polimorfizmus által gyűjtő típusnak lett kitalálva.
- Haszna főként akkor van, ha szeretnénk létrehozni egy ősosztályt, ami nem példányosítható, úgy, hogy mindeközben funkcionális követelményeket állíthatunk, melyeket a leszármazott osztályoknak kötelezően implementálniuk kell.

Absztrakt osztály II.

- Tehát az absztrakt osztályt a sima osztályoktól eltérően nem lehet példányosítani, hiszen kizárólag örököltetési célokból hozzuk létre.
- Az absztrakt osztály absztrakt alprogrammal is rendelkezhet, melyeket az ős csak előír, de nem implementál. Az előírt metódusokat a leszármazottak kötelesek megvalósítani.
- Normál osztály nem rendelkezhet absztrakt alprogrammal.

Absztrakt osztály III.

- Példa egy absztrakt osztály használatára:



- Ebben a hierarchiában a *RepuloDolgok* osztály egyedüli feladata, hogy közös őst szolgáltasson az alá tartozó osztályoknak, adott esetben rájuk kényszerítve a repul() alprogramot.

Absztrakt osztály IV.

- Így a *RepuloDolgok*, a programon belül önállóan sehol nem jöhet létre, továbbá a repul() alprogram implementálását sem tudja maga megoldani, hiszen a madár, a repülő, és a labda is másképpen repül, így azt absztraktnak hozzuk létre.
- Létrehozása:
  - `abstract class <Név>`

Absztrakt osztály V.

- Szerkezete:
  - Felépítését tekintve, pontosan ugyanolyan formában kell megvalósítani, mint egy normál osztályt, azonban azokat az alprogramokat melyeket az osztály nem implementál, abstract előtaggal kell létrehozni.
    - `<láthatóság> abstract <visszatérés> <Név>(<paraméterek>);`
  - Példa:
    - `public abstract void orokoltMetodus(String in);`

Absztrakt osztály VI.

- Mindazonáltal absztrakt osztály is tartalmazhat nem absztrakt programkódot, tehát absztrakt osztálynak is lehet az osztályhoz hasonló alprogramja, mely teljesen implementálva van.
- Habár példányosítani nem lehet (kívülről), de a leszármazottakban való felhasználásakor implicit „példányosodik”.

Absztrakt osztály VII.

- Absztrakt osztály az absztrakt alprogramot úgy tudja használni, mintha az implementálva lenne.
- A fordító -mivel példányosítani nem lehet, a leszármazottnak pedig implementálnia kell az alprogramot- „megelőlegzi” az implementálást.
- Absztrakt osztályból származtatott osztály is lehet absztrakt.
- A leszármazottban implementált –az ősben eredetileg absztrakt- alprogram a további öröklődésben felülírható.

Kivételkezelés I.

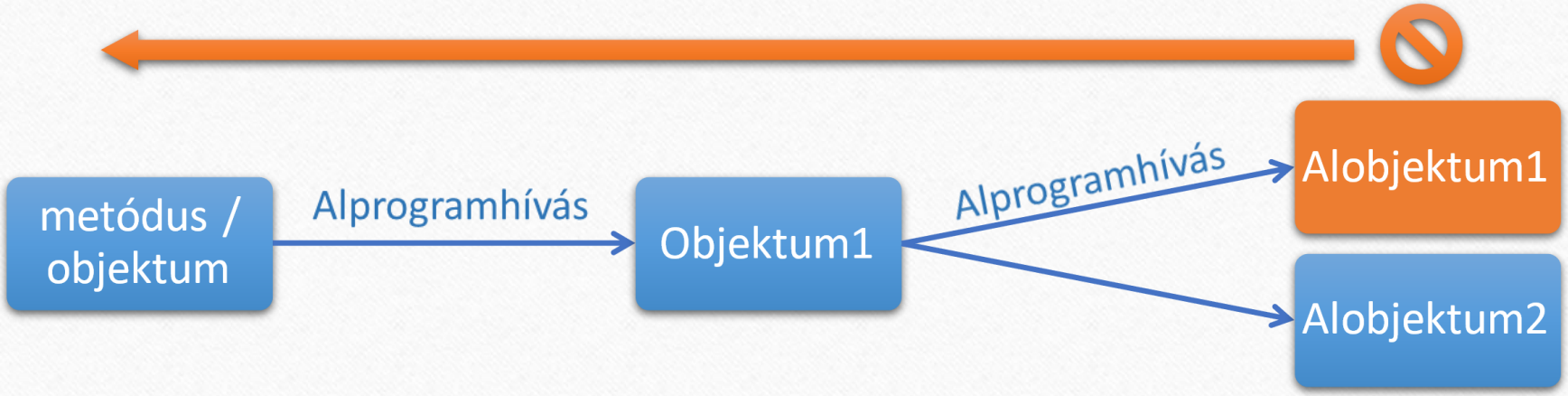
- Kivételnek nevezzük azt az eseményt, amikor a program, futása során hibába ütközik, melyet nem tud kezelni, és ebből adódóan a program futása megakad.
- A hibák lehetnek a kódunk szemantikai hibáinak eredményei, de természetesen van rajtunk kívül álló okokból, vagy csak közvetve miattunk bekövetkező hiba is.

Kivételkezelés II.

- Azonban ezek a kivételek szerencsére kezelhetők, mivel a lehetséges kivételeket el tudjuk kapni, és tudunk rá programkód szinten reagálni.
- Egy teljesen hibatűrő program az összes kivételt kezelné valahogyan, így nincs olyan dolog ami kiakaszthatja, azonban általánosságban ez nem jellemző, hiszen a hibák kivédésére szolgáló programkód hosszabb lenne, mint maga a program.

Kivételkezelés III.

- A kivételek terjedése, mindig a hívó kód felé halad, tehát mindig az adott hibához vezető hívási soron megy visszafelé:



Kivételkezelés IV.

- A kivételkezelés formája:
  - ```
    try
    {
       //kivételkezelt kód
    }
    catch ( KivételOsztálya változó ) //specifikus kivétel
    {
       //ha bekövetkezik…
    }
    catch ( Exception változó )
    {
       //minden más, nem specifikusan kezelt kivétel esetén
    }
    finally
    {
       //végezetül… (opcionális)
    }
    ```
    ```

Kivételkezelés V.

- A fő kérdés, hogy honnan tudjuk, hogy melyik alprogram, milyen kivételt okozhat
  - A dokumentáció -amennyiben megfelelően megírták- segít, mivel az alprogram kiválasztásakor kiírja, hogy milyen kivételt válthat ki az adott alprogram.
  - Próbálkozás alapján → futtatjuk és amilyen kivételt dob azt kezeljük le.
  - Használhatjuk a polimorfizmust is, az egyes kivételek ugyanis egymásból származ(hat)nak.

Kivételkezelés VI.

- A kivételek két nagy csoportra bonthatók:
  - Ellenőrzött kivételek → nem futási időben keletkezett kivételek. A fordító kikényszeríti a hiba kezelését.
    - Tipikusan input/output műveletek kivételei tartoznak ide.
  - Futási idejű kivételek → A Java futtatórendszerében (JRE) keletkező kivételek.
    - Ide tartoznak többek között a referencia kivételek, aritmetikai kivételek, indexeléssel kapcsolatos kivételek, argumentumokkal kapcsolatos kivételek.

Tiszta kód V.

- A hibakezelés fontos, de nem dominálhat a programban
  - Hibakódok használata helyett kivételeket (hibadobás) alkalmazzunk!
  - Készítsünk beszédes hibaüzenetet, amit adjunk át a kivétellel együtt.
  - Készítsünk olyan teszteseteket, amik kiváltják a kivételeket és a kivételkezelők a tesztek elvárásaihoz igazodnak.
  - Akkor használjunk eltérő kivételosztályokat, ha bizonyos típusúakat el akarunk fogni, a többieket pedig továbbadjuk.

Kivételkezelés VII.

- Kivételdobásra akkor van szükségünk, ha egy objektumon belül olyan hiba adódik, ami az objektum szempontjából végzetes, azonban nyelvi szempontokból az adott művelet semmilyen akadályba nem ütközik.
- Egyedi kivételkezelés további előnye, hogy az osztályon belül belső tesztelésekkel állapítjuk meg a hiba lehetséges formáját, a hiba kezelését azonban már a feldolgozóra hagyjuk.

Kivételkezelés VIII.

- A művelet nagyban meg tudja könnyíteni a fejlesztés menetét, hiszen, az osztályon belül nekünk nem kell bajlódnunk azzal, hogy maga az osztály példánya kezelje le a hibákat.
- Természetesen elkerülhetetlen, hogy az esetlegesen felmerülő kivételeket, melyeket nem maga az objektum készít el és dob, azt továbbra is kezelnünk kell.

Kivételkezelés IX.

- Egyedi kivételkezelés szintaxisa:
  - throw new Exception(<paraméterek>);
  - Mint az látható, maga a kivétel is egy objektum, melyet mi eldobunk. Amennyiben azt egy try-catch elkapja, tudunk rá reagálni.
  - Amennyiben nem kapja el, a program hibaüzenettel leáll.

Feladat II.

- Írjon alkalmazást a mellékelt „Összefoglaló projektmunka.pdf”-ben található specifikáció alapján.